

Stack Blur and Random Walks: Length-Generalization Failure in Differentiable Stacks

Anonymous authors
Paper under double-blind review

Abstract

XXX

1 Introduction

MM:

- Can a model learn to generalize beyond its training horizon?
- Learning algorithms is a sensible test case: they have length invariant rules which the model has to infer from training data!
- Length generalization failure can occur for a number of reasons, notably due to memory fidelity issues.
- Differentiable stacks attempt to externalize memory, hopefully getting rid of the memory fidelity problem. However, memory fidelity issues still arise due to uncertainty in memory actions which leads to blurring.
- In this work we quantitatively characterize the effect of this blurring. For a minimal LIFO memory benchmark sequence reversal and under a simple stochastic controller abstraction we provide a ballot-counting argument to derive a retrieval-fidelity decay bound.

2 Background and Related Work

MM:

- Differentiable stacks and neural memory. Describe how neural stacks and Turing machines etc work, define them and cite the right references.
- Review works studying their properties, in particular commentary on length generalization in algorithmic tasks.

3 Fixed Error Analysis via Ballot Counting

In this section, we analyse the memory fidelity of an abstract probabilistic stack controller solving a minimal exact-retrieval task with a constant error rate. The main goal is computing a bound on the expected symbol we output, and show that it decays towards random guessing with the length of the string.

3.1 Problem Formulation and Stack Indicators

Our main focus is the algorithmic task of binary string reversal, which relies heavily on the stack mechanism. Let Γ be a finite alphabet of tokens (e.g., a binary vocabulary where $|\Gamma| = 2$). The model receives an input sequence of length N , denoted as $\xi(1), \dots, \xi(N)$, where $\xi(u) \in \Gamma$. Following an encoding phase where we push the input sequence onto the stack, a phase switch occurs at time $\tau = N$, after which the model outputs the exact sequence in reverse order.

The model uses an internal last-in-first-out (LIFO) memory structure modeled via an abstract probabilistic stack controller. At each timestep t , the controller executes a discrete action: **push** during the encoding phase ($t \leq \tau$) and **pop** during the decoding phase ($t > \tau$). In our abstract controller, we formalise this by defining a fixed error rate $\epsilon \in (0, 0.5)$. At any timestep t , the controller executes the correct action with probability $1 - \epsilon$, and the incorrect action with probability ϵ .

To formalize the trajectory of tokens, we define the following indicators:

Definition 1 (Stack Indicators). *Let $m_k(t)$ represent the element at depth k of the stack at time t , with the top of the stack given as $m_1(t)$.*

- Let $\lambda(t, u)$ denote the position on the stack of the u -th object seen in the input buffer.
- Let $B_u(t) := \mathbf{1}(\lambda(t, u) = 1)$ denote the event that the element seen at time u in the input sequence is at the top of the stack. The events $\{B_u(t)\}_{u \in \mathbb{N}}$ are mutually disjoint.
- Let $\omega(t, k) := \mathbf{1}(m_k(\tau) \text{ is at the top of the stack at time } t)$ denote the event that an element that was at depth k at the phase switch time τ is at the top of the stack at time t .

Definition 2 (Algorithmic Fidelity vs. Symbol Correctness). *Suppose a specific target token is pushed onto the stack at time $t_i \leq \tau$, and must be retrieved at output time $t_{out} = 2\tau - t_i + 1$. We define two types of retrieval probabilities:*

- **Algorithmic Fidelity:** *The probability that the exact token pushed onto the stack at t_i is at the top of the stack at t_{out} , given by $\mathbb{P}(\lambda(t_{out}, t_i) = 1)$.*
- **Symbol Correctness:** *The probability that $m_1(t_{out})$ shares the same symbol $\gamma \in \Gamma$ as the target $\xi(t_i)$, regardless of whether it is the true target token or not.*

Using these definitions, the total probability mass for the correct symbol γ being at the top of the stack can be decomposed into two sums, representing probability mass coming from elements on the stack when we switch phases, and new elements erroneously read in the pop phase:

$$\mathbb{P}(m_1(t) = \gamma) = \sum_{k=1}^{n+1} \mathbb{P}(\omega(t, k) = 1) \mathbb{P}(m_k(\tau) = \gamma) + \sum_{\tau \leq u \leq t} \mathbb{P}(\lambda(t, u) = 1) \mathbb{P}(\xi(u) = \gamma),$$

where $n = \tau - t_i$ is the distance from the phase switch. Our primary objective is to bound the Algorithmic Fidelity.

3.2 Stack Dynamics as a Bounded Random Walk

We model the trajectory of a specific token on the stack as a 1-dimensional integer random walk. The number of timesteps from the insertion of the token to the phase switch is $n = \tau - t_i$. For string reversal, this is also the number of steps from the phase switch to the output time ($n = t_{out} - \tau$).

Lemma 3 (Stack combinatorics). *The number of valid integer random walks consisting of n steps of unit length, starting at depth 1 and ending at depth k , strictly avoiding the boundary at 0 is given by*

$$N_{1 \rightarrow k, n} = \binom{n}{\frac{n+k-1}{2}} - \binom{n}{\frac{n+k+1}{2}}.$$

For $N_{1 \rightarrow k, n}$ to be non-zero, n and $k - 1$ must share the same parity.

Phase 1: Encoding. In the first phase, we push onto the stack with probability $1 - \epsilon$ and pop off from it with probability ϵ . Let $U = \frac{n+k-1}{2}$ denote the number of pushes, and $D = \frac{n-k+1}{2}$ the number of pops. By Lemma 3, the probability that the symbol seen in the input at time t_i is at depth k at τ is:

$$\mathbf{P}_{enc}(k) = \left[\binom{n}{\frac{n+k-1}{2}} - \binom{n}{\frac{n+k+1}{2}} \right] (1 - \epsilon)^{\frac{n+k-1}{2}} \epsilon^{\frac{n-k+1}{2}}.$$

Phase 2: Decoding. We now push with probability ϵ and pop with probability $1 - \epsilon$. The number of valid paths from k to 1 in n steps is given by $N_{k \rightarrow 1, n}$. Due to the symmetry of the binomial coefficient, this is exactly equal to the number of paths in the encoding phase. Thus, the probability of the element at depth k at time τ returning to the top of the stack is identical to the encoding probability:

$$\mathbf{P}_{enc}(k) = \mathbf{P}_{dec}(k) = \mathbf{P}(k).$$

3.3 Theoretical Bounds on Memory Fidelity

By extending the definition of our encoding and decoding probabilities to allow for an arbitrary distance d from the delimiter time τ , denoted $\mathbf{P}_{enc}(k, d)$ and $\mathbf{P}_{dec}(k, d)$, we can fully decompose the total symbol correctness probability into three terms:

$$\begin{aligned} \mathbb{P}(m_1(t) = \gamma) &= \underbrace{\sum_{k=1}^{n+1} \mathbf{P}_{dec}(k, n) \mathbf{P}_{enc}(k, n)}_{\text{Term 1: Algorithmic Fidelity}} \\ &+ \underbrace{\sum_{k=1}^{n+1} \mathbf{P}_{dec}(k, n) \sum_{j \neq t_i} \mathbf{P}_{enc}(k, N - j) \mathbb{P}(\xi(j) = \gamma)}_{\text{Term 2: Correct symbols from erroneous times}} \\ &+ \underbrace{\sum_{\tau \leq u \leq t} \mathbb{P}(\lambda(t, u) = 1) \mathbb{P}(\xi(u) = \gamma)}_{\text{Term 3: Errors during decoding phase}}. \end{aligned}$$

Theorem 4 (Decay of Memory Fidelity). *For an abstract probabilistic stack controller with a constant error rate $\epsilon \in (0, 0.5)$, the expected algorithmic fidelity (Term 1) of a token pushed onto the stack n timesteps before the phase switch is strictly bounded above by $\mathcal{O}(1/\sqrt{n})$:*

$$\text{Algorithmic Fidelity} \leq \frac{1}{\sqrt{2en\epsilon(1-\epsilon)}}.$$

Proof. Let $m = \frac{n+k-1}{2}$. Noting that $\mathbf{P}_{enc}(k, n) = \mathbf{P}_{dec}(k, n) = \mathbf{P}(k)$, the algorithmic fidelity evaluates to the sum of squared probabilities over all valid intermediate stack depths k :

$$\text{Term 1} = \sum_{k=1}^{n+1} [\mathbf{P}(k)]^2 \leq \left(\max_k \mathbf{P}(k) \right) \sum_{k=1}^{n+1} \mathbf{P}(k) \leq \max_k \mathbf{P}(k),$$

where the inequality holds because the events $\{\mathbf{P}(k)\}_{k \in [1, n+1]}$ are mutually exclusive, meaning their sum is at most 1.

We can bound $\mathbf{P}(k)$ by including the paths that drop below zero. Doing so yields the strict inequality:

$$\mathbf{P}(k) < \binom{n}{m} (1 - \epsilon)^m \epsilon^{n-m}.$$

□

3.4 Total Symbol Correctness and Convergence to Random Guessing

To rigorously establish the convergence of the total symbol correctness to random guessing, we explicitly take the expectation over the input sequence. We assume the input sequence ξ is drawn uniformly at random from the binary alphabet Γ^N , such that for any index $j \neq t_i$ or $u \geq \tau$, the probability of an incidental match with the target symbol is $\mathbb{P}(\xi(j) = \gamma) = \mathbb{P}(\xi(u) = \gamma) = \frac{1}{2}$.

Let us isolate the path traversal probabilities from the symbol matching probabilities. We define the pure path probabilities as follows:

- $\mathcal{P}_{target} := \sum_{k=1}^{n+1} \mathbf{P}_{dec}(k, n) \mathbf{P}_{enc}(k, n)$ as the probability of retrieving the exact target token.
- $\mathcal{P}_{collision} := \sum_{k=1}^{n+1} \mathbf{P}_{dec}(k, n) \sum_{j \neq t_i} \mathbf{P}_{enc}(k, N - j)$ as the probability of retrieving a token pushed during the encoding phase at an incorrect time.
- $\mathcal{P}_{error} := \sum_{\tau \leq u \leq t} \mathbb{P}(\lambda(t, u) = 1)$ as the probability of retrieving a token erroneously pushed during the decoding phase.

Because the controller must read a token at the output time t (assuming $\epsilon < 0.5$ keeps the stack bounded away from underflow with high probability), these three events partition the valid stack trajectories. Thus, their probabilities sum to one:

$$\mathcal{P}_{target} + \mathcal{P}_{collision} + \mathcal{P}_{error} = 1.$$

Taking the expectation of the total symbol correctness over the uniform data distribution ξ , we have:

$$\begin{aligned} \mathbb{E}_{\xi}[\mathbb{P}(m_1(t) = \gamma)] &= \mathcal{P}_{target} \cdot \mathbb{P}(\xi(t_i) = \gamma \mid \xi(t_i) = \gamma) \\ &\quad + \mathcal{P}_{collision} \cdot \mathbb{P}(\xi(j) = \gamma) \\ &\quad + \mathcal{P}_{error} \cdot \mathbb{P}(\xi(u) = \gamma). \end{aligned}$$

Since the retrieved target token is guaranteed to match itself (probability 1), and all other tokens are independent and uniformly distributed (probability 1/2), this simplifies to:

$$\mathbb{E}_{\xi}[\mathbb{P}(m_1(t) = \gamma)] = \mathcal{P}_{target} + \frac{1}{2}(\mathcal{P}_{collision} + \mathcal{P}_{error}).$$

Substituting the identity $\mathcal{P}_{collision} + \mathcal{P}_{error} = 1 - \mathcal{P}_{target}$, we obtain:

$$\mathbb{E}_{\xi}[\mathbb{P}(m_1(t) = \gamma)] = \mathcal{P}_{target} + \frac{1}{2}(1 - \mathcal{P}_{target}) = \frac{1}{2} + \frac{1}{2}\mathcal{P}_{target}.$$

By Theorem 4, the algorithmic fidelity $\mathcal{P}_{target}$ is bounded above by $\mathcal{O}(1/\sqrt{n})$. Therefore, as the distance n from the phase switch increases, the expected probability of reading the correct symbol strictly decays to random guessing:

$$\begin{aligned} \mathbb{E}_{\xi}[\mathbb{P}(m_1(t) = \gamma)] &\leq \frac{1}{2} + \mathcal{O}\left(\frac{1}{\sqrt{n}}\right). \\ \lim_{n \rightarrow \infty} \mathbb{E}_{\xi}[\mathbb{P}(m_1(t) = \gamma)] &= \frac{1}{2}. \end{aligned}$$

4 Simulation Study and Model Diagnostics

To empirically validate the bounds derived above, we simulate abstract stack trajectories and investigate the properties of trained models.

MM:

- Can we use online statistics (e.g., measuring the ϵ rates during inference) and feed them into a dynamic program to estimate our confidence dynamically?
- Discuss the pros and cons of hardening stack actions to mitigate the soft stack blurring.

Figure 1: Validation of the Upper Bound on Memory Fidelity.

Figure 2: Convergence of Symbol Correctness to Random Guessing ($p = 0.5$) over sequence length.

5 Summary, Limitations and Future Work

MM: To do...

Reproducibility Statement: Code able to reproduce our experimental results can be found in the following anonymous repository, [MM: INSERT ONCE DONE](#).

Ethics Statement: This work uses only synthetic, non-sensitive data, involves no human or animal subjects, and carries minimal dual-use or environmental risks.

Broader Impact Statement: This work contributes to the mathematical and conceptual foundations of machine learning and intelligent systems. A better understanding of the underlying principles governing learning, generalization, and representation can inform the long-term development of more reliable and interpretable algorithms.

A Appendix

XXX

Bence: TODO by friday:

- Construct a diagnostic that is based on the entropy of the stack to measure the amount of blurring.
- Can this be used to tell whether we learned the actual algorithm or just a shortcut based on the rate of decay after the training horizon?
- Select a suite of tasks from the Deletang paper to demonstrate this behaviour.